
Documentação de Projeto

para o sistema

Elo

Versão 1.0

Projeto de sistema elaborado pela aluna Laura Campos Pontara Lopes
como parte da disciplina **Projeto de Software**.

09 de junho de 2026

Tabela de Conteúdo

1. Introdução	1
1.1 Contextualização	1
2. Modelos de Usuário e Requisitos	1
2.1 Descrição de Atores	1
2.2 Modelo de Casos de Uso e Histórias de Usuários	2
2.2.1 Histórias de Usuário	4
2.3 Diagrama de Sequência do Sistema e Contrato de Operações	7
2.3.1 UC13: Comprar Ingresso	7
2.3.2 UC08: Criar Evento com Lotes	9
2.3.3 UC17: Realizar Check-in Digital	11
3. Modelos de Projeto	13
3.1 Arquitetura	13
3.2 Diagrama de Componentes e Implantação	14
3.3 Diagrama de Classes	16
3.4 Diagramas de Sequência	17
3.5 Diagramas de Comunicação	20
3.6 Diagramas de Estados	23
4. Modelos de Dados	24

Histórico de Revisões

Nome	Data	Razões para Mudança	Versão
Entrega 1	09/06/2026	Criação do documento. Inclusão das seções 1 a 4: contextualização, modelagem de usuários e requisitos, modelos de projeto (arquitetura, componentes, implantação, classes, sequência, comunicação e estados) e modelo de dados do sistema Elo.	1.0

1. Introdução

Este documento agrega: 1) a elaboração e revisão de modelos de domínio e 2) modelos de projeto para o sistema Elo. A referência principal para a descrição geral do problema, do domínio e dos requisitos do sistema é o documento de especificação que descreve a visão de domínio do sistema.

1.1 Contextualização

A gestão de comunidades por *creators* e *community managers* ainda depende de um conjunto fragmentado de ferramentas: uma planilha para controlar membros, uma plataforma de eventos para vender ingressos, um provedor de e-mail para campanhas e ferramentas avulsas para mensurar engajamento. Essa fragmentação gera retrabalho, perda de informação entre sistemas, dificuldade de consolidar métricas e uma experiência inconsistente para o membro da comunidade.

O **Elo** é uma plataforma SaaS de gestão de comunidades que reúne, em um único ecossistema, todas as operações de um *creator*: criação e administração de comunidades, publicação de eventos com venda de ingressos, gestão de membros, campanhas de comunicação, check-in digital no dia do evento e acompanhamento da saúde da comunidade por meio de *insights* em tempo real. A plataforma é organizada em duas superfícies:

- **Gestor Elo:** painel privado onde o *creator* gerencia comunidades, eventos, membros, campanhas e insights;
- **Checkout Elo:** aplicação pública para listagem de eventos, RSVP e compra de ingressos com pagamento via PIX e cartão de crédito.

O nome “Elo” é fictício e utilizado para fins acadêmicos, preservando a identidade da empresa; a arquitetura aqui documentada é baseada em um sistema real em produção, mas foi remodelada para representar uma arquitetura de referência adequada aos requisitos do domínio.

2. Modelos de Usuário e Requisitos

Esta seção apresenta os modelos que capturam quem utiliza o sistema Elo e o que se espera dele. Primeiro são descritos os atores que interagem com a plataforma; em seguida, o modelo de casos de uso acompanhado das histórias de usuário que detalham os requisitos funcionais; e, por fim, os diagramas de sequência do sistema com os respectivos contratos de operação para os casos de uso mais representativos do domínio.

2.1 Descrição de Atores

Nesta subseção é apresentada a descrição de cada um dos atores que interagem com o sistema Elo. Os atores humanos especializam o ator genérico **Usuário**; os atores de sistema

representam serviços externos que colaboram com a plataforma.

Ator	Tipo	Descrição
Usuário	Humano (genérico)	Ator base que representa qualquer pessoa autenticada na plataforma. Concentra as capacidades comuns de autenticação.
Creator (Gestor)	Humano	Organizador da comunidade. Cria e administra comunidades, eventos, lotes de ingressos, campanhas, realiza o check-in e consulta insights. É o ator principal da superfície Gestor.
Membro	Humano	Pessoa que participa de uma comunidade. Solicita entrada, faz RSVP em eventos e recebe notificações e campanhas.
Comprador	Humano	Pessoa que adquire ingressos pela superfície Checkout. Pode atuar como convidado (<i>guest</i>), sem conta prévia.
Gateway de Pagamento	Sistema	Serviço externo que processa cobranças via PIX e cartão de crédito e notifica a confirmação por <i>webhook</i> .
Serviço de E-mail (Resend)	Sistema	Serviço externo de envio transacional de e-mails (ingressos, confirmações e campanhas).
Supabase Auth	Sistema	Provedor de identidade responsável pela emissão e validação de tokens JWT.

2.2 Modelo de Casos de Uso e Histórias de Usuários

Nesta subseção é apresentado o diagrama de casos de uso do sistema. Cada caso de uso possui um identificador (UC-XX) que serve de referência cruzada no restante do documento. O diagrama de casos de uso é exibido na Figura 1, em orientação paisagem para melhor legibilidade.

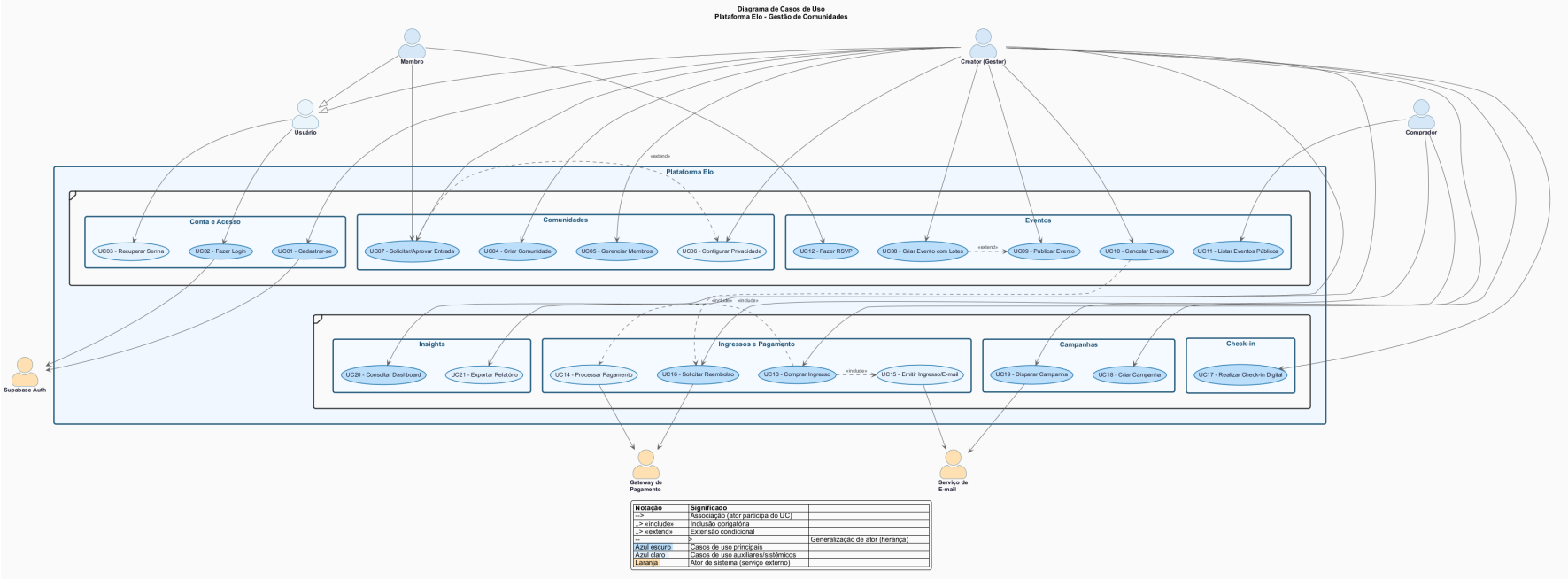


Figura 1: Diagrama de Casos de Uso da plataforma Elo.

A Tabela a seguir relaciona cada caso de uso ao seu ator principal e à(s) história(s) de usuário correspondente(s).

ID	Caso de Uso	Ator Principal	História
UC01	Cadastrar-se na Plataforma	Creator	US01
UC02	Fazer Login	Usuário	US02
UC03	Recuperar Senha	Usuário	US03
UC04	Criar Comunidade	Creator	US04
UC05	Gerenciar Membros	Creator	US05
UC06	Configurar Privacidade	Creator	US06
UC07	Solicitar/Aprovar Entrada	Membro/Creator	US07
UC08	Criar Evento com Lotes	Creator	US08
UC09	Publicar Evento	Creator	US09
UC10	Cancelar Evento	Creator	US10
UC11	Listar Eventos Públicos	Comprador	US11
UC12	Fazer RSVP	Membro	US12
UC13	Comprar Ingresso	Comprador	US13
UC14	Processar Pagamento	Gateway de Pag.	US13
UC15	Emitir Ingresso e Enviar E-mail	Serviço E-mail	US14
UC16	Solicitar Reembolso	Comprador	US15
UC17	Realizar Check-in Digital	Creator	US16
UC18	Criar Campanha	Creator	US17
UC19	Disparar Campanha	Creator	US18
UC20	Consultar Dashboard de Insights	Creator	US20
UC21	Exportar Relatório de Insights	Creator	US21

2.2.1 Histórias de Usuário

A seguir são descritas as histórias de usuário do sistema Elo, no formato “Como / Quero / Para que”, acompanhadas de seus critérios de aceitação.

US01: Cadastrar-se na Plataforma. **Como** *creator*, **quero** criar uma conta informando nome, e-mail e senha, **para que** eu possa gerenciar minhas comunidades. *Critérios:* (i) e-mail deve ser único; (ii) a senha deve atender à política mínima de segurança; (iii) a conta é criada via Supabase Auth e um e-mail de confirmação é enviado.

US02: Fazer Login. **Como** usuário cadastrado, **quero** autenticar-me com e-mail e senha, **para que** eu acesse as funcionalidades do meu perfil com segurança. *Critérios:* (i) credenciais inválidas exibem mensagem genérica, sem revelar qual campo está incorreto; (ii) após o login é emitido um JWT; (iii) sessões inativas por mais de 30 minutos são encerradas.

US03: Recuperar Senha. **Como** usuário cadastrado, **quero** recuperar o acesso via e-mail, **para que** eu redefina minha senha caso a esqueça. *Critérios:* (i) o link de redefinição tem validade de 1 hora; (ii) após o uso, o link é invalidado; (iii) o sistema não confirma nem nega a existência do e-mail.

US04: Criar Comunidade. Como *creator*, **quero** criar uma comunidade com nome, descrição e identificador único (*slug*), **para que** eu tenha um espaço para organizar membros e eventos. *Critérios:* (i) o *slug* deve ser único; (ii) o criador é registrado como OWNER; (iii) a comunidade nasce com privacidade padrão configurável.

US05: Gerenciar Membros. Como *creator*, **quero** promover, rebaixar e remover membros, **para que** eu controle os papéis dentro da comunidade. *Critérios:* (i) apenas OWNER/ADMIN podem gerenciar membros; (ii) o OWNER não pode ser removido; (iii) membros removidos perdem o acesso, mas o histórico é preservado.

US06: Configurar Privacidade. Como *creator*, **quero** definir a comunidade como pública ou privada, **para que** eu controle quem pode entrar e visualizar conteúdo. *Critérios:* (i) comunidades privadas exigem aprovação de entrada; (ii) comunidades públicas permitem entrada imediata.

US07: Solicitar/Aprovar Entrada. Como membro, **quero** solicitar entrada em uma comunidade e, como *creator*, aprovar ou recusar a solicitação, **para que** o acesso seja controlado. *Critérios:* (i) em comunidade privada a solicitação fica PENDENTE; (ii) a aprovação muda o status para ATIVO e notifica o membro.

US08: Criar Evento com Lotes. Como *creator*, **quero** criar um evento e definir lotes de ingressos com preço e quantidade, **para que** eu possa vendê-los. *Critérios:* (i) somente OWNER/ADMIN criam eventos; (ii) o evento nasce em RASCUNHO; (iii) cada lote possui período de vendas válido (início anterior ao fim).

US09: Publicar Evento. Como *creator*, **quero** publicar um evento, **para que** ele fique visível no Checkout. *Critérios:* (i) o evento deve ter ao menos um lote e dados obrigatórios completos; (ii) ao publicar, recebe PUBLICADO e uma URL pública.

US10: Cancelar Evento. Como *creator*, **quero** cancelar um evento, **para que** ele saia do ar quando não puder ocorrer. *Critérios:* (i) o status muda para CANCELADO; (ii) pedidos pagos são marcados para reembolso; (iii) os participantes são notificados.

US11: Listar Eventos Públicos. Como comprador, **quero** visualizar os eventos publicados, **para que** eu escolha em qual participar. *Critérios:* (i) apenas eventos PUBLICADO aparecem; (ii) cada evento exibe lotes e saldo disponível.

US12: Fazer RSVP. Como membro, **quero** confirmar presença em um evento gratuito, **para que** eu garanta minha vaga. *Critérios:* (i) o RSVP respeita a capacidade do evento; (ii) o membro recebe confirmação por e-mail.

US13: Comprar Ingresso. Como comprador, **quero** comprar ingressos pagando com PIX ou cartão, **para que** eu participe de um evento pago. *Critérios:* (i) a compra só prossegue se houver saldo no lote; (ii) o pedido nasce PENDENTE e só é confirmado após o pagamento; (iii) o saldo do lote é baixado na emissão dos ingressos.

US14: Receber Ingresso por E-mail. Como comprador, **quero** receber meus ingressos por e-mail com QR Code, **para que** eu apresente na entrada. *Critérios:* (i) o e-mail é enviado após a confirmação do pagamento; (ii) cada ingresso possui um QR Code único.

US15: Solicitar Reembolso. Como comprador, **quero** solicitar o reembolso de um pedido pago, **para que** eu recupere o valor quando não puder comparecer. *Critérios:* (i) apenas pedidos PAGO podem ser reembolsados; (ii) o reembolso é solicitado ao gateway; (iii) os ingressos são cancelados.

US16: Realizar Check-in Digital. Como *creator*, **quero** validar ingressos por QR Code na entrada, **para que** eu controle o acesso ao evento. *Critérios:* (i) QR inválido ou de outro evento é recusado; (ii) ingresso já utilizado é sinalizado; (iii) ingresso válido é marcado como UTILIZADO com data/hora.

US17: Criar Campanha. Como *creator*, **quero** criar campanhas de comunicação para a comunidade, **para que** eu engaje meus membros. *Critérios:* (i) a campanha define canal (e-mail/push) e segmento; (ii) nasce em RASCUNHO; (iii) pode ser agendada para envio futuro.

US18: Disparar Campanha. Como *creator*, **quero** disparar uma campanha, **para que** a mensagem chegue ao segmento escolhido. *Critérios:* (i) o envio é assíncrono via fila; (ii) somente membros do segmento recebem; (iii) o status muda para ENVIADA ao concluir.

US19: Receber Notificações. Como membro, **quero** receber notificações sobre eventos e campanhas, **para que** eu acompanhe a comunidade. *Critérios:* (i) o canal padrão é o e-mail cadastrado; (ii) as notificações ficam registradas para consulta.

US20: Consultar Dashboard de Insights. Como *creator*, **quero** consultar métricas de engajamento, crescimento e saúde da comunidade, **para que** eu tome decisões com base em dados. *Critérios:* (i) os indicadores são atualizados periodicamente; (ii) o painel cobre membros, eventos e vendas.

US21: Exportar Relatório de Insights. Como *creator*, **quero** exportar os insights em PDF ou CSV, **para que** eu compartilhe os resultados. *Critérios:* (i) o relatório reflete o estado no momento da exportação; (ii) o CSV contém as colunas das métricas principais.

2.3 Diagrama de Sequência do Sistema e Contrato de Operações

Nesta subseção são apresentados os diagramas de sequência do sistema (*caixa-preta*) de três casos de uso representativos do domínio Elo (**UC13 Comprar Ingresso**, **UC08 Criar Evento** e **UC17 Check-in**), seguidos dos contratos das principais operações de sistema identificadas em cada um.

2.3.1 UC13: Comprar Ingresso

Neste caso de uso, o comprador seleciona os lotes desejados, informa seus dados e efetua o pagamento por PIX ou cartão de crédito. A confirmação do pagamento ocorre de forma assíncrona, após o retorno do gateway, quando os ingressos são emitidos e enviados por e-mail. O diagrama de sequência do sistema é exibido na Figura 2.

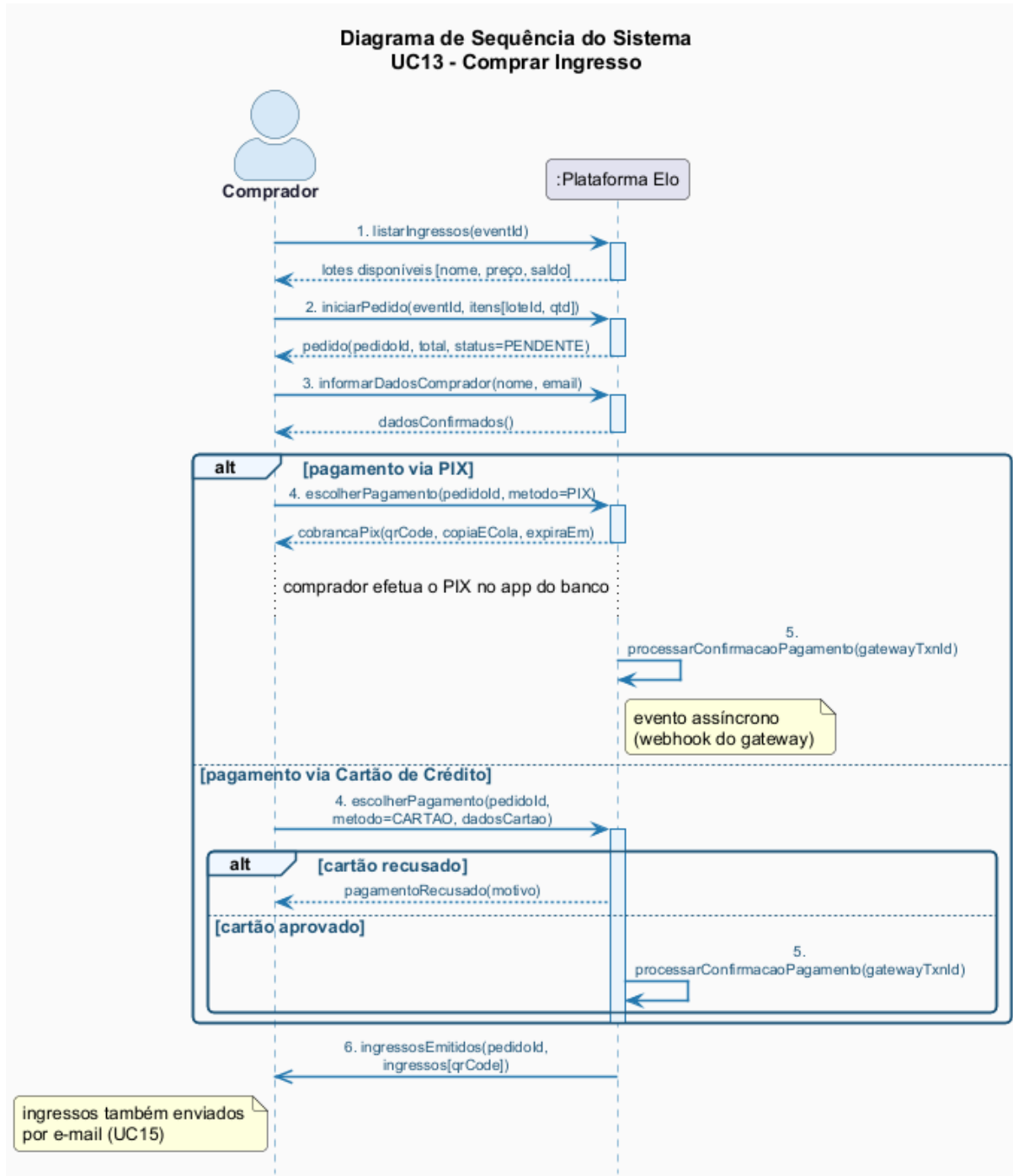


Figura 2: Diagrama de Sequência do Sistema: UC13 Comprar Ingresso.

Contrato	CO1: Iniciar Pedido
Operação	iniciarPedido(eventId, itens[loteId, qtd])
Referências cruzadas	Caso de Uso UC13: Comprar Ingresso; História US13.
Pré-condições	O evento existe e está PUBLICADO; os lotes informados pertencem ao evento.
Pós-condições	Um Pedido foi criado com status PENDENTE; os ItemPedido foram associados ao pedido; o total foi calculado.

Contrato	CO2: Confirmar Pagamento
Operação	processarConfirmacaoPagamento(gatewayTxnId)
Referências cruzadas	Caso de Uso UC13/UC14; Histórias US13, US14.
Pré-condições	Existe um Pagamento PENDENTE associado ao gatewayTxnId; o gateway confirmou a transação.
Pós-condições	O Pagamento e o Pedido passaram a PAG0; um job de emissão de ingressos foi enfileirado; os ingressos são emitidos e enviados por e-mail.

2.3.2 UC08: Criar Evento com Lotes

Neste caso de uso, o *creator* informa os dados do evento, adiciona um ou mais lotes de ingressos e, por fim, publica o evento, tornando-o visível no Checkout. O diagrama de sequência do sistema é exibido na Figura 3.

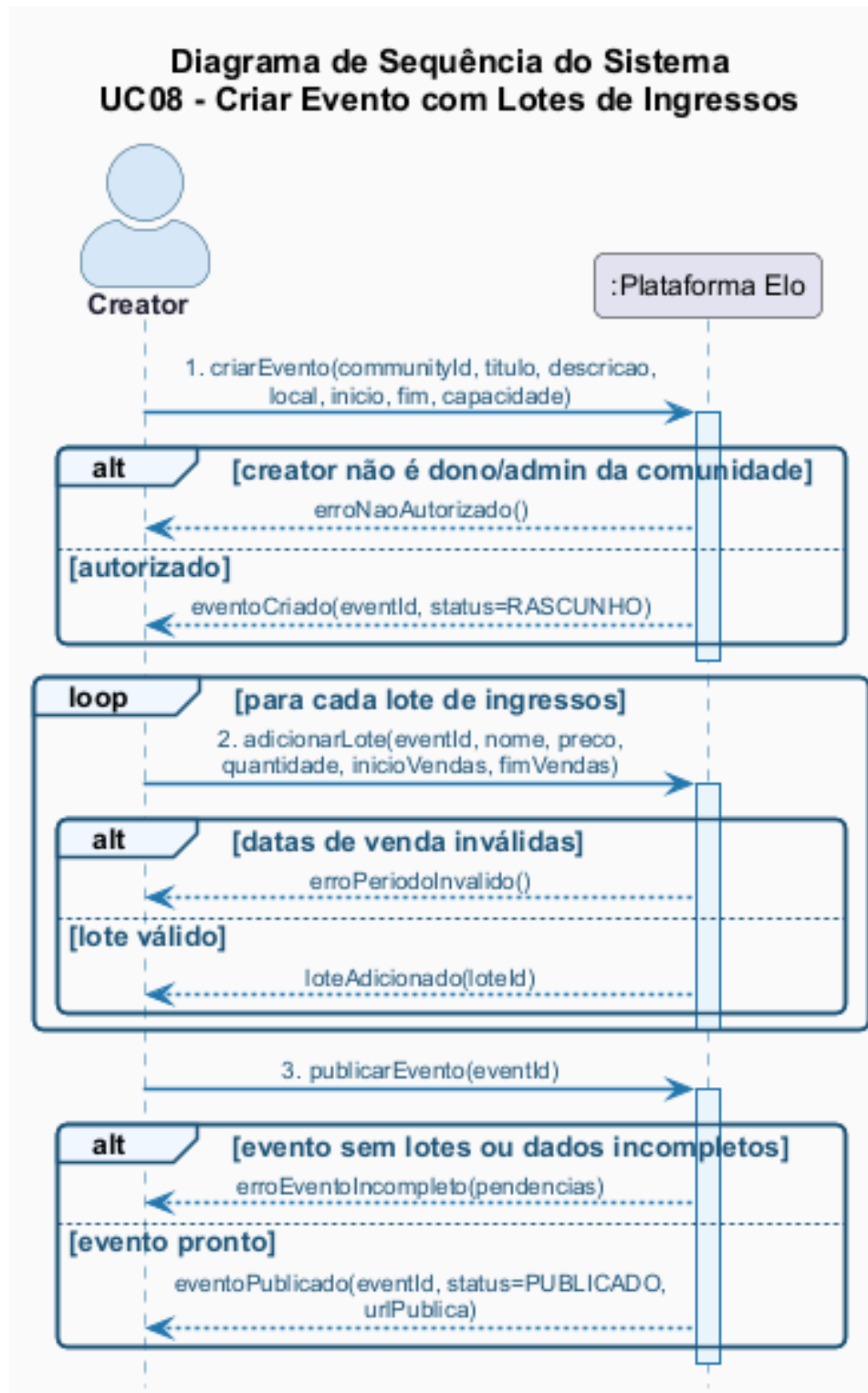


Figura 3: Diagrama de Sequência do Sistema: UC08 Criar Evento.

Contrato	CO3: Criar Evento
Operação	<code>criarEvento(communityId, dados)</code>
Referências cruzadas	Caso de Uso UC08; História US08.
Pré-condições	O usuário autenticado é OWNER ou ADMIN da comunidade.
Pós-condições	Um Evento foi criado com status RASCUNHO, vinculado à comunidade.

Contrato	CO4: Publicar Evento
Operação	<code>publicarEvento(eventId)</code>
Referências cruzadas	Caso de Uso UC09; História US09.
Pré-condições	O evento está em RASCUNHO, possui ao menos um lote e dados obrigatórios completos.
Pós-condições	O evento passou a PUBLICADO e recebeu uma URL pública; tornou-se visível no Checkout.

2.3.3 UC17: Realizar Check-in Digital

Neste caso de uso, na entrada do evento, o *creator* valida o QR Code de cada ingresso apresentado. O sistema verifica se o ingresso é válido, se já foi utilizado ou se é inválido, registrando o check-in quando apropriado. O diagrama de sequência do sistema é exibido na Figura 4.

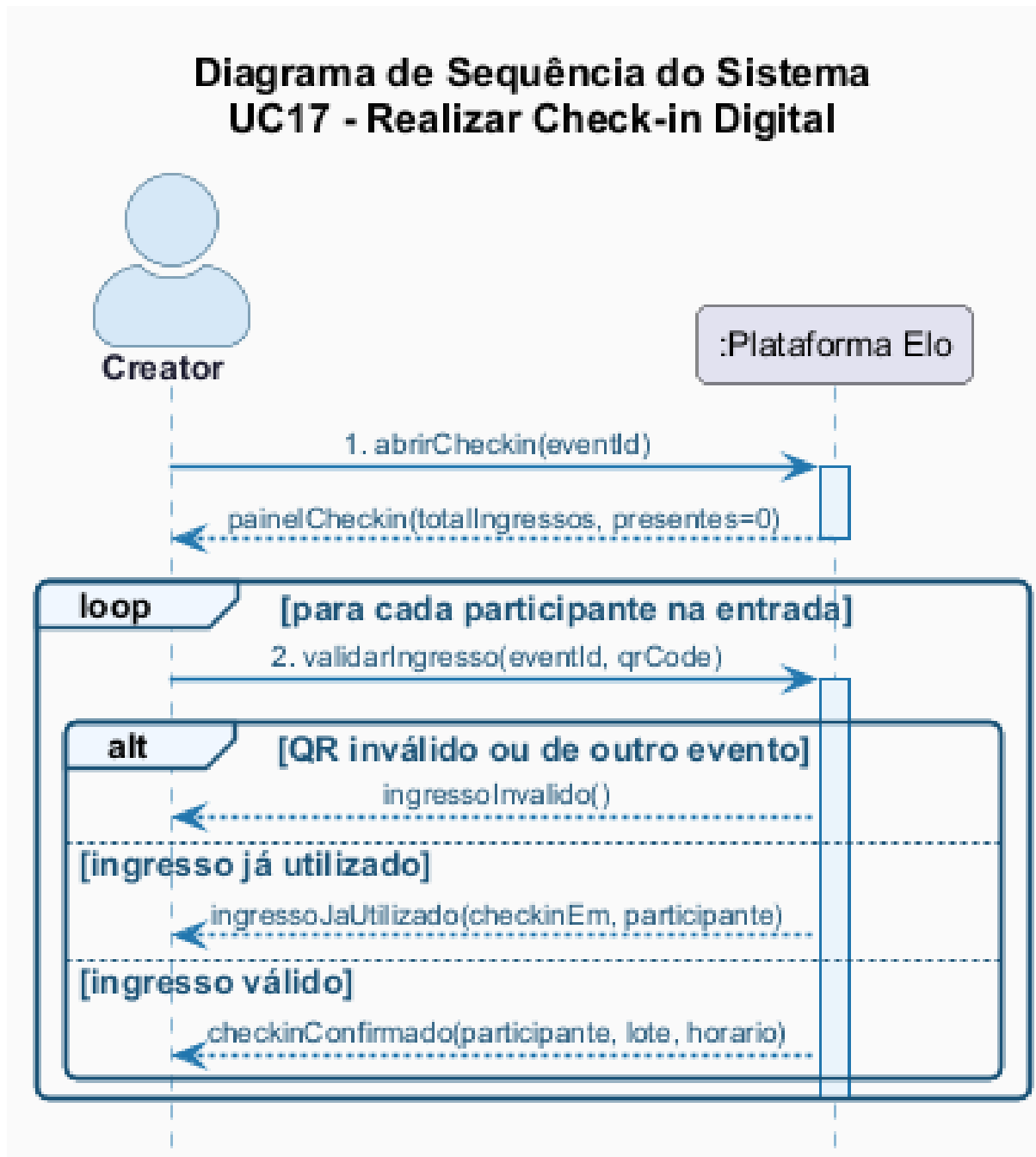


Figura 4: Diagrama de Sequência do Sistema: UC17 Check-in.

Contrato	CO5: Validar Ingresso
Operação	validarIngresso(eventId, qrCode)
Referências cruzadas	Caso de Uso UC17; História US16.
Pré-condições	Existe um ingresso correspondente ao qrCode; o <i>creator</i> tem permissão no evento.
Pós-condições	Se o ingresso era VALIDO, passou a UTILIZADO com data/hora de check-in registrada; caso contrário, o sistema sinaliza “inválido” ou “já utilizado” sem alterar o estado.

3. Modelos de Projeto

Esta seção detalha as decisões de projeto que realizam os requisitos apresentados na Seção 2. São descritos a arquitetura geral da solução, a organização interna em componentes e sua implantação física, o modelo de classes do domínio e os diagramas dinâmicos (sequência, comunicação e estados) que especificam o comportamento do sistema na realização dos casos de uso.

3.1 Arquitetura

A arquitetura do Elo é descrita pelo modelo **C4** no nível de contêineres (Nível 2), exibido na Figura 5. Vale destacar que, no modelo C4, um *contêiner* representa uma unidade executável e implantável de forma independente (uma aplicação, um serviço ou um banco de dados), e não um contêiner Docker. Cada superfície (Gestor e Checkout) é uma aplicação Next.js independente que consome uma API REST central (Elo API, em Fastify). A API encapsula as regras de negócio em uma *Service Layer*, persiste dados em PostgreSQL (Supabase) por meio do padrão *Repository* (Drizzle ORM) e delega o trabalho assíncrono (envio de e-mails, campanhas e processamento de confirmação de pagamento) a um **Worker** acionado por uma fila (Cloud Tasks). A identidade é provida pelo Supabase Auth (JWT), e serviços externos (Gateway de Pagamento e Resend) integram-se via HTTPS e *webhooks*.

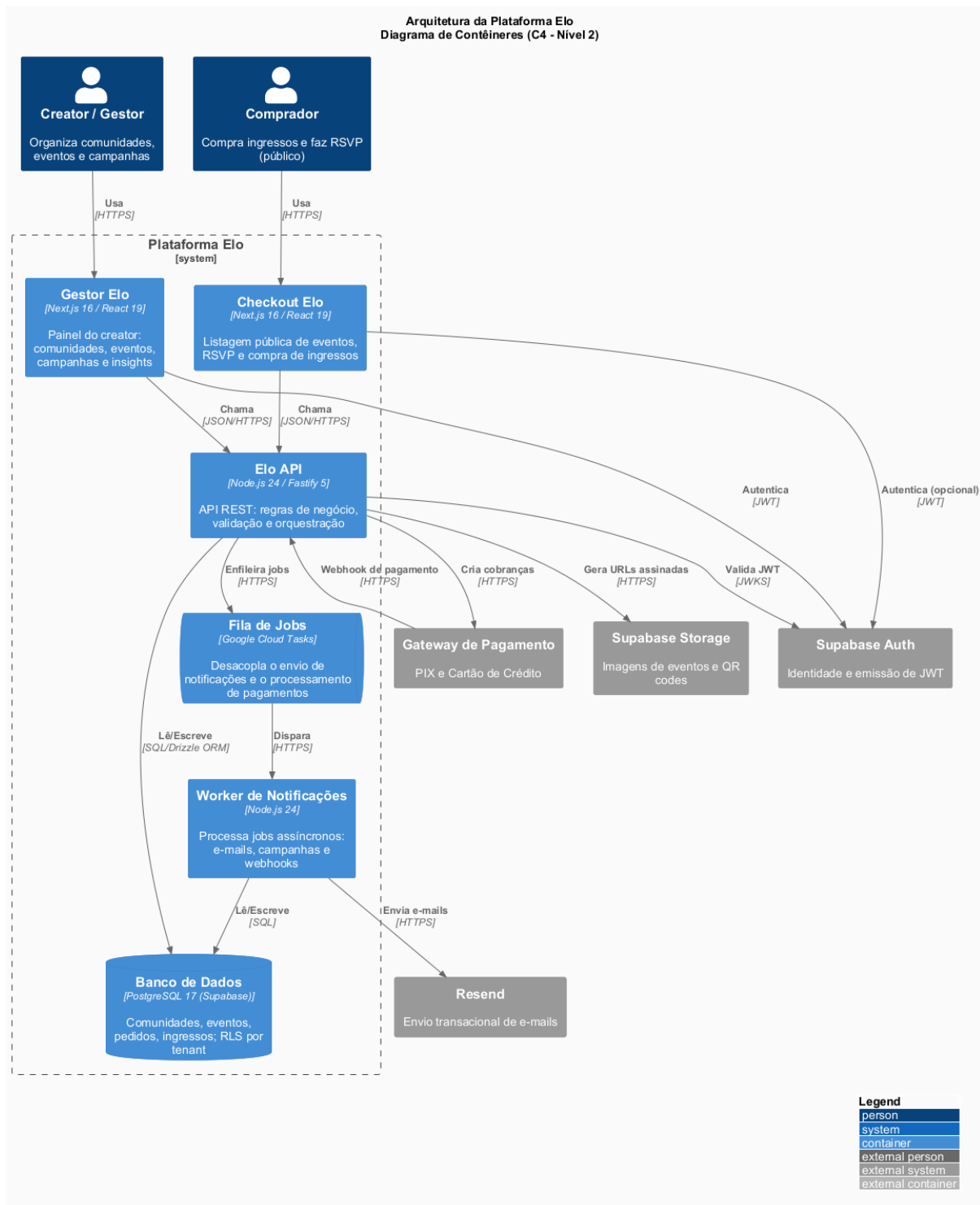


Figura 5: Diagrama de Contêineres (C4, Nível 2) da plataforma Elo.

3.2 Diagrama de Componentes e Implantação

O diagrama de componentes (Figura 6) detalha a organização interna da Elo API: a camada HTTP (roteamento, autenticação JWT e validação com Zod), os módulos de negócio (*services*) e suas interfaces fornecidas e requeridas, a camada de acesso a dados (*repositories*) e os adaptadores de saída para gateway de pagamento, e-mail, *storage* e fila.

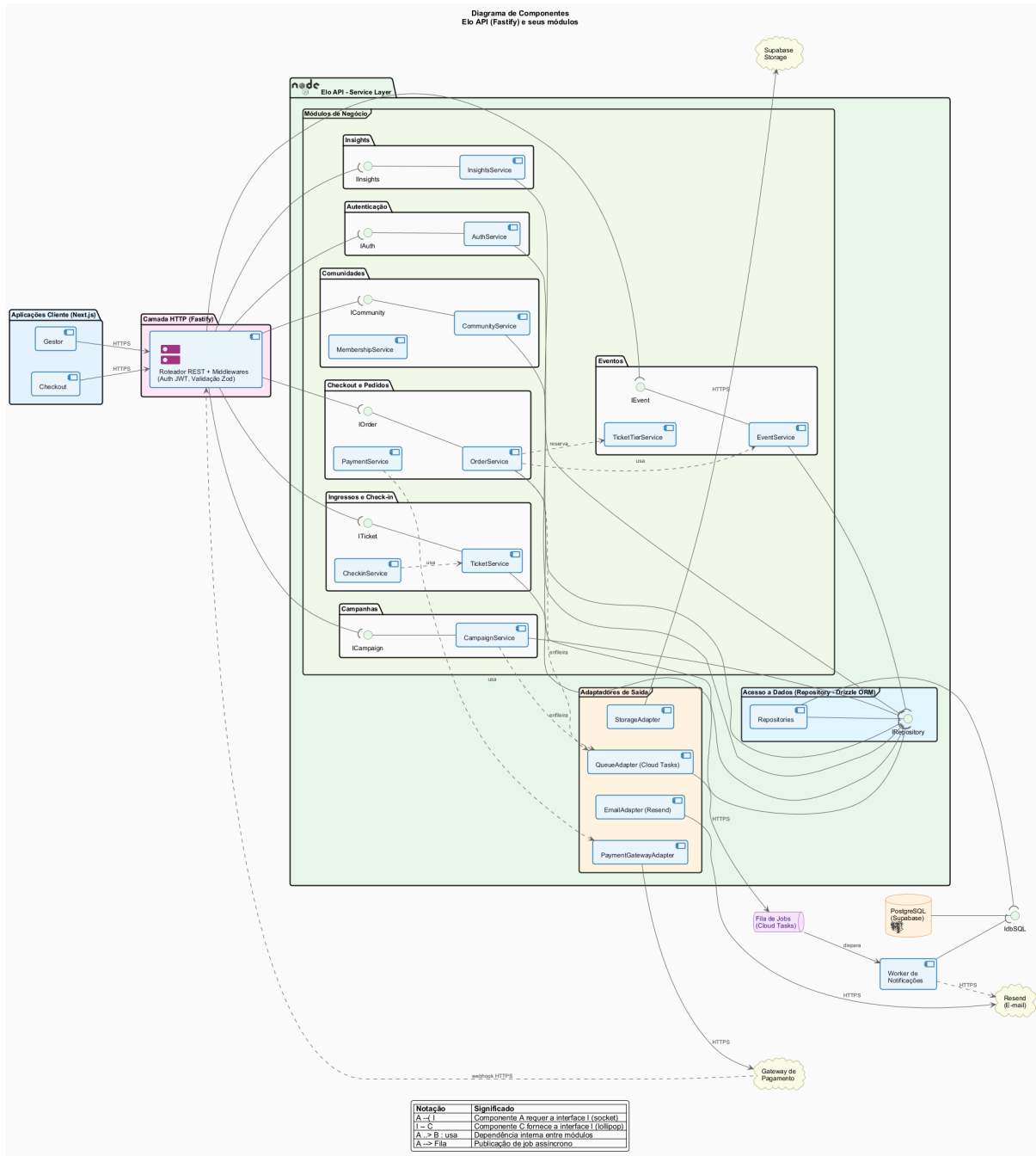


Figura 6: Diagrama de Componentes da Elo API.

O diagrama de implantação (Figura 7) mostra a alocação física dos componentes: os front-ends Next.js hospedados na Vercel; a Elo API e o Worker como contêineres no Google Cloud Run; a fila no Cloud Tasks; o banco PostgreSQL, a autenticação e o *storage* no Supabase; e os serviços SaaS externos (Gateway de Pagamento e Resend). O fluxo de CI/CD é operado pelo GitHub Actions.

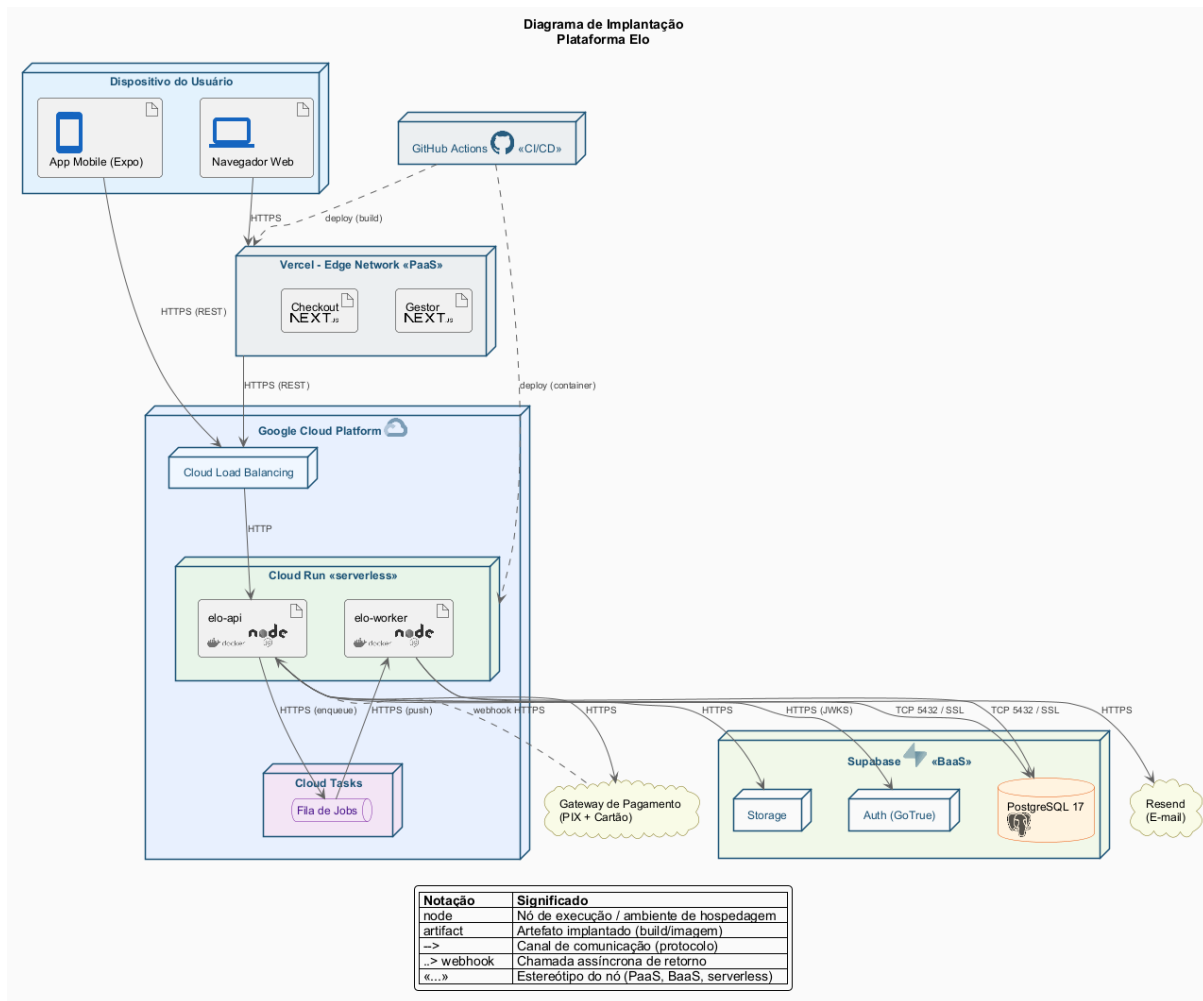


Figura 7: Diagrama de Implantação da plataforma Elo.

3.3 Diagrama de Classes

A Figura 8 apresenta o modelo de domínio do Elo. As entidades centrais são Comunidade, Evento, Lote, Pedido, Pagamento e Ingresso, conectadas por relações de composição e associação. As enumerações modelam os estados e categorias do domínio (status de evento, pedido e ingresso, papéis de membro, métodos de pagamento e canais de campanha).

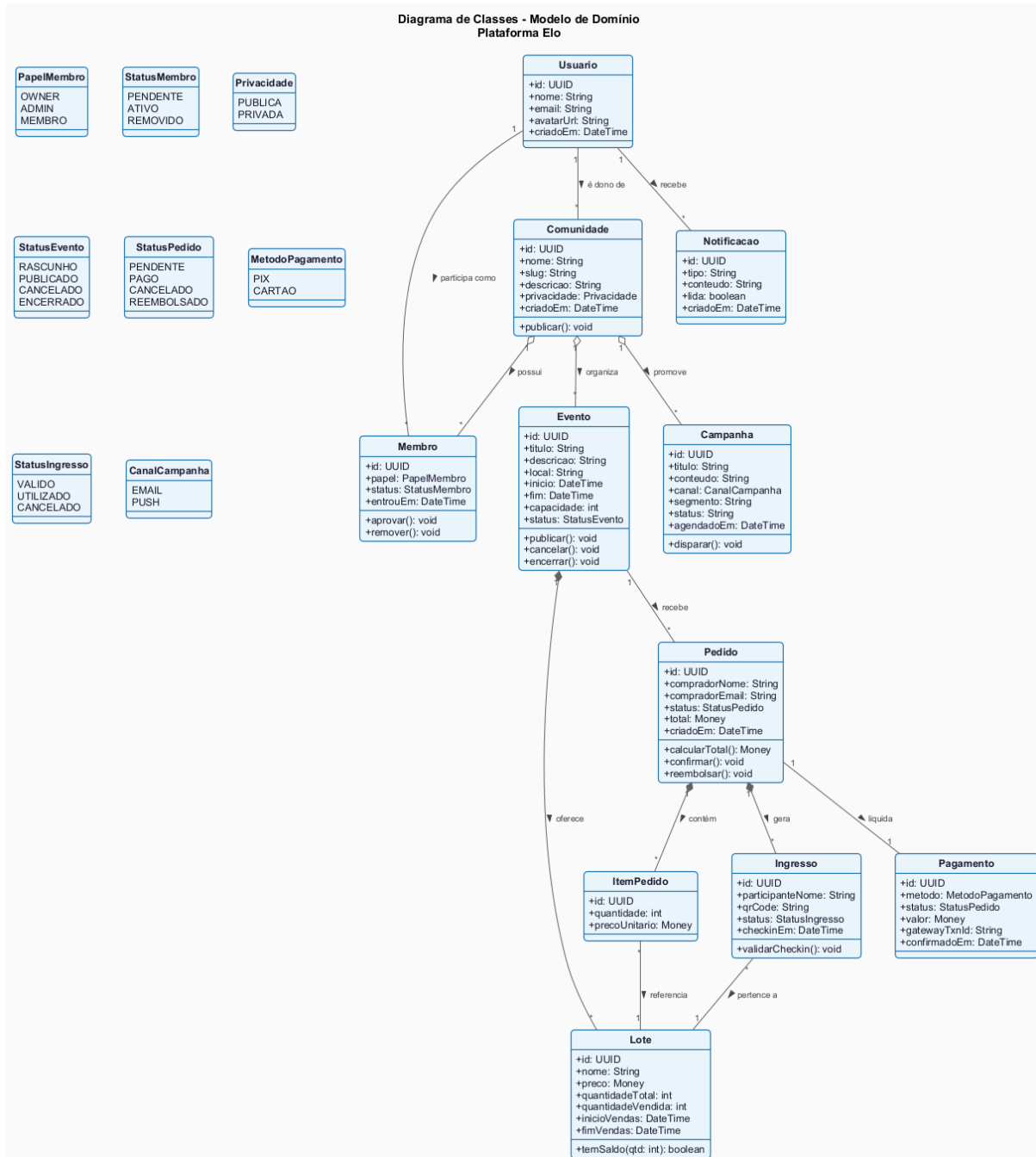


Figura 8: Diagrama de Classes: modelo de domínio da plataforma Elo.

3.4 Diagramas de Sequência

Os diagramas a seguir realizam, em nível de projeto (*caixa-branca*), os três casos de uso selecionados, detalhando a colaboração entre as fronteiras de interface (*boundary*), os controladores/serviços (*control*) e o repositório de dados (*database*).

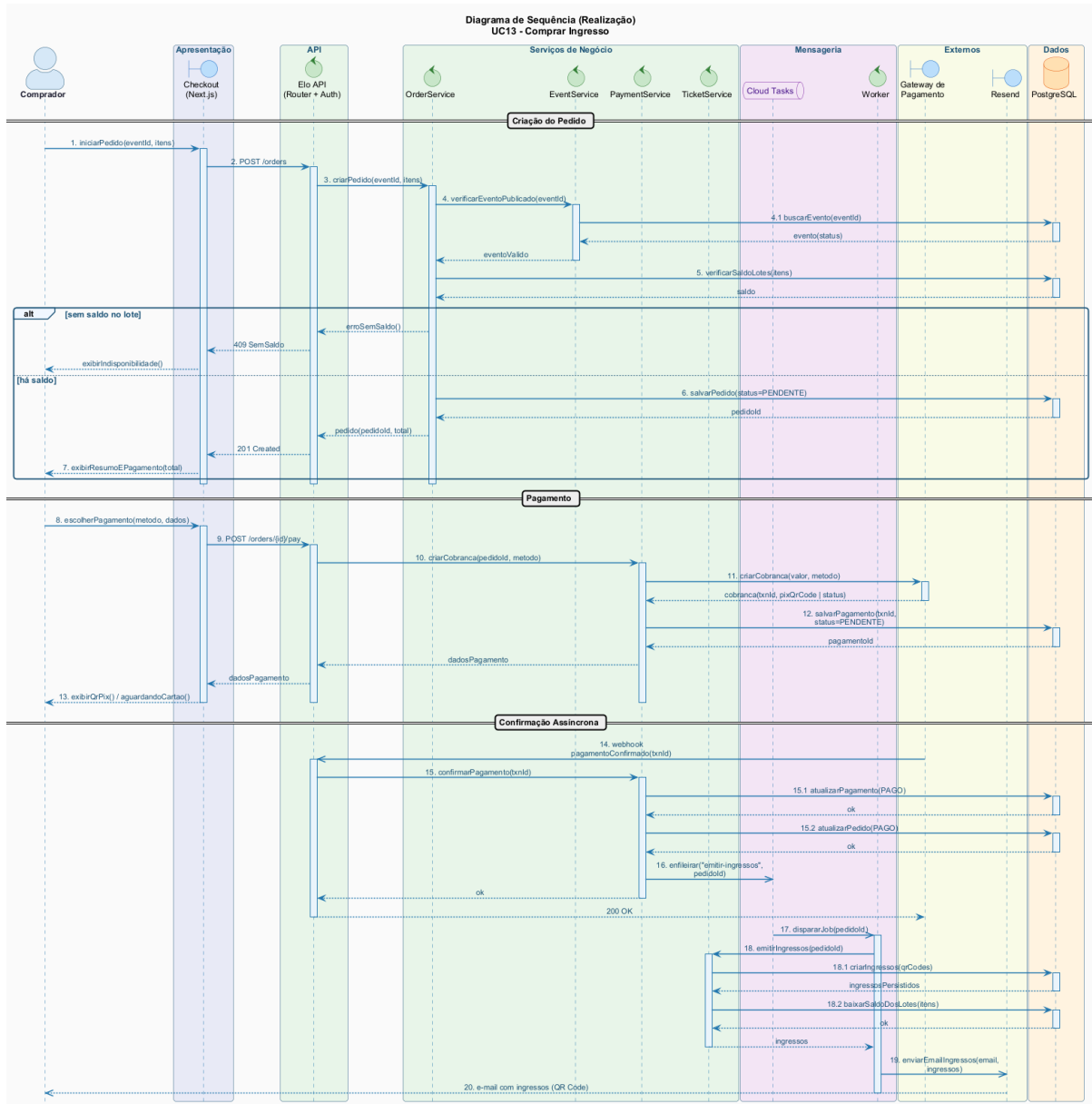


Figura 9: Diagrama de Sequência: realização do UC13 Comprar Ingresso.

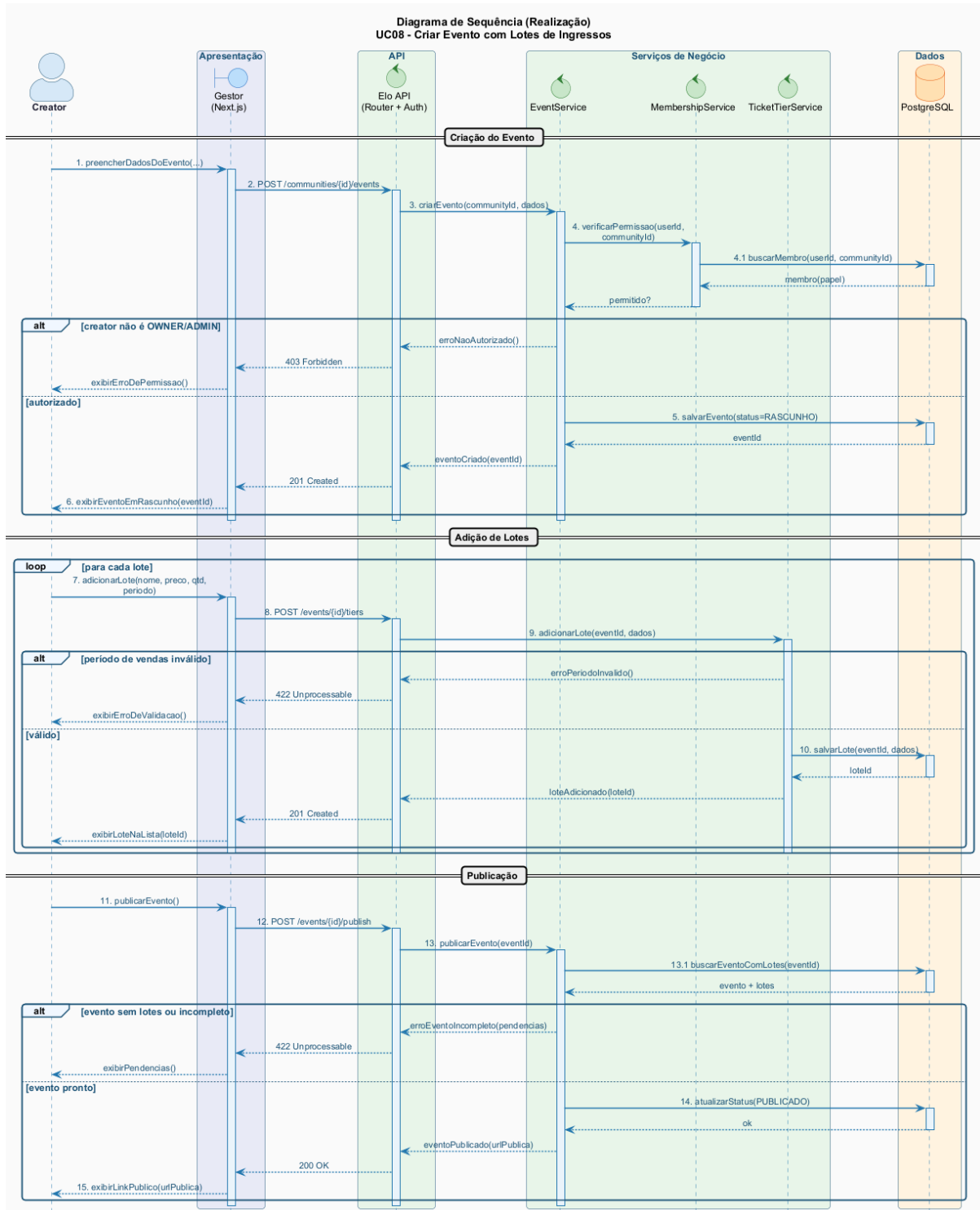


Figura 10: Diagrama de Sequência: realização do UC08 Criar Evento.

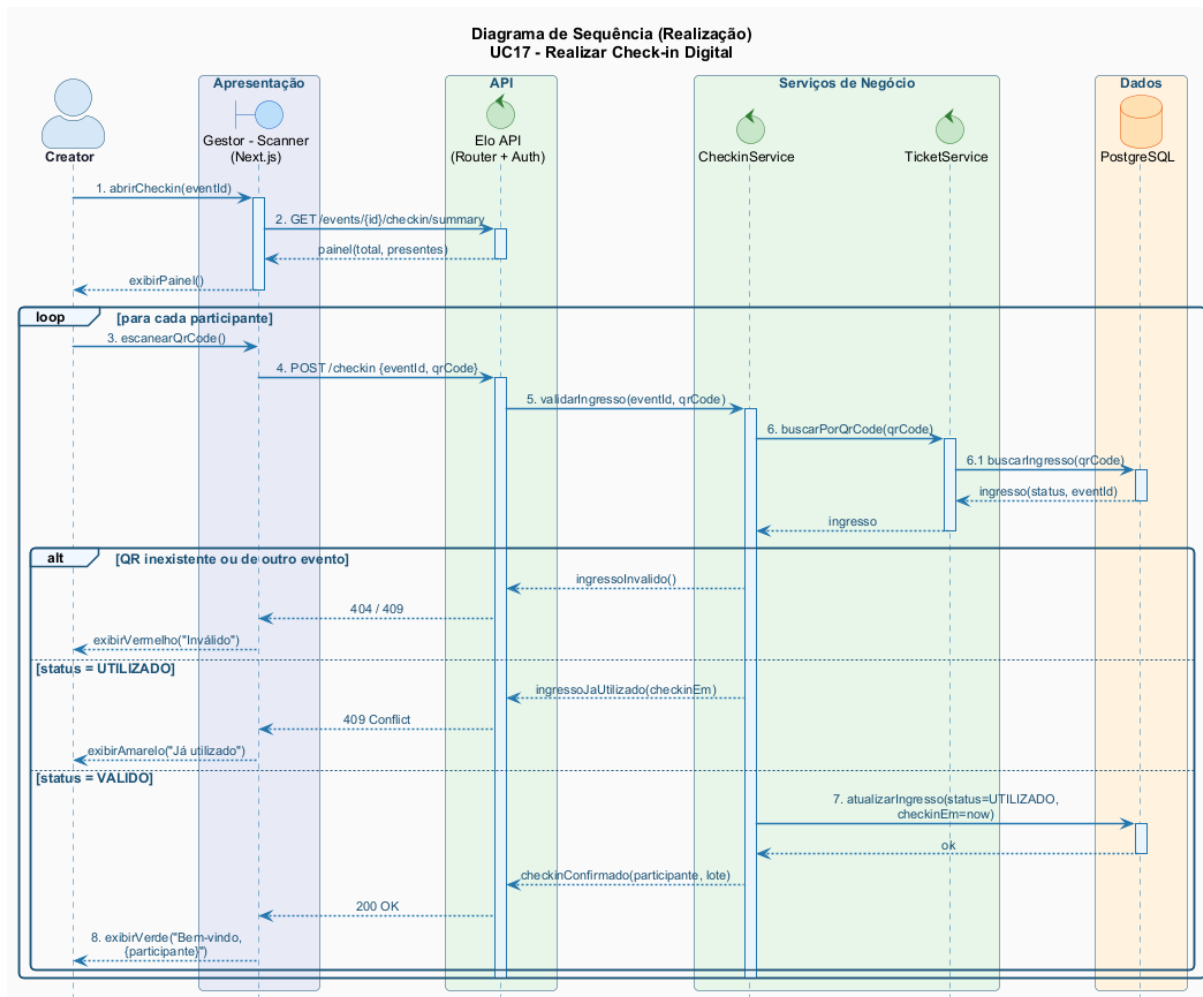


Figura 11: Diagrama de Sequência: realização do UC17 Check-in.

3.5 Diagramas de Comunicação

Os diagramas de comunicação representam as mesmas interações dos diagramas de sequência, enfatizando os vínculos entre objetos colaboradores e a ordem temporal das mensagens (numeração). São apresentados para os três casos de uso selecionados: Comprar Ingresso (Figura 12), Criar Evento (Figura 13) e Check-in (Figura 14).

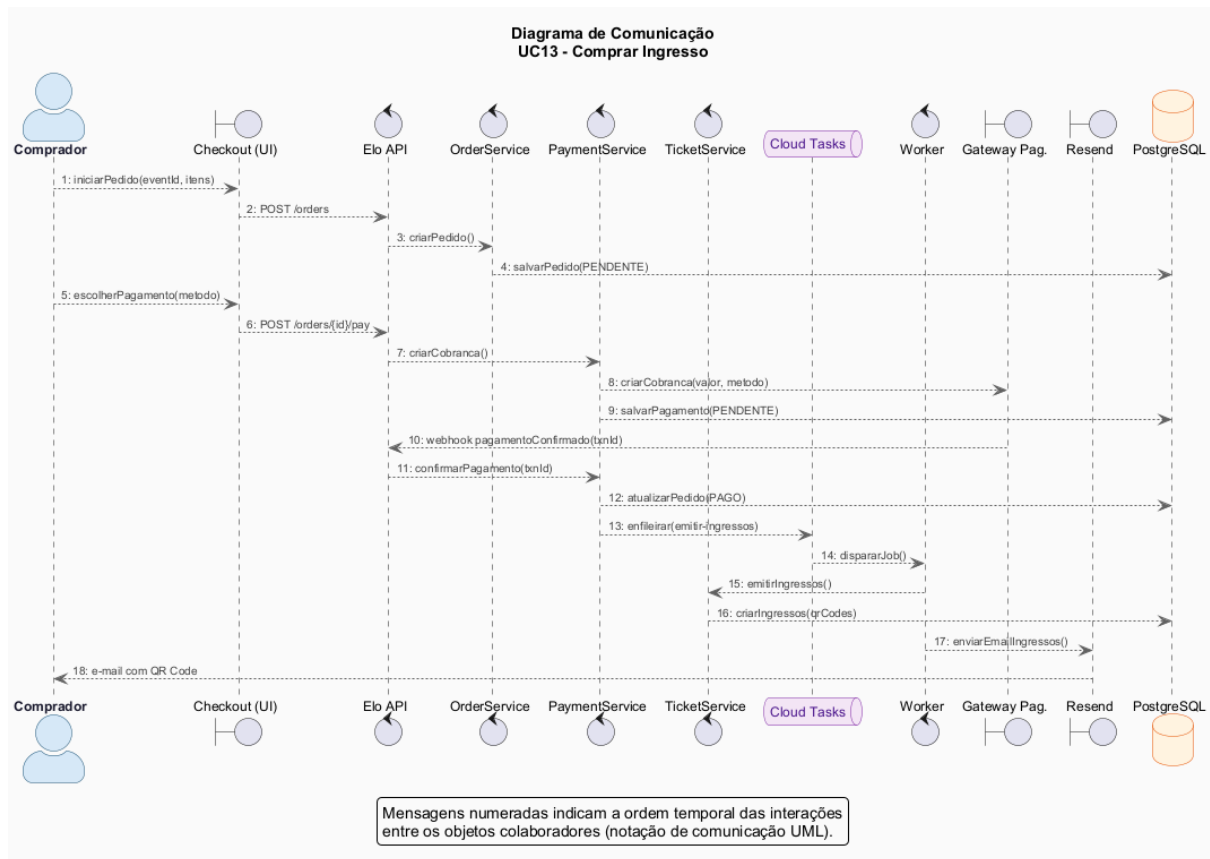


Figura 12: Diagrama de Comunicação: UC13 Comprar Ingresso.

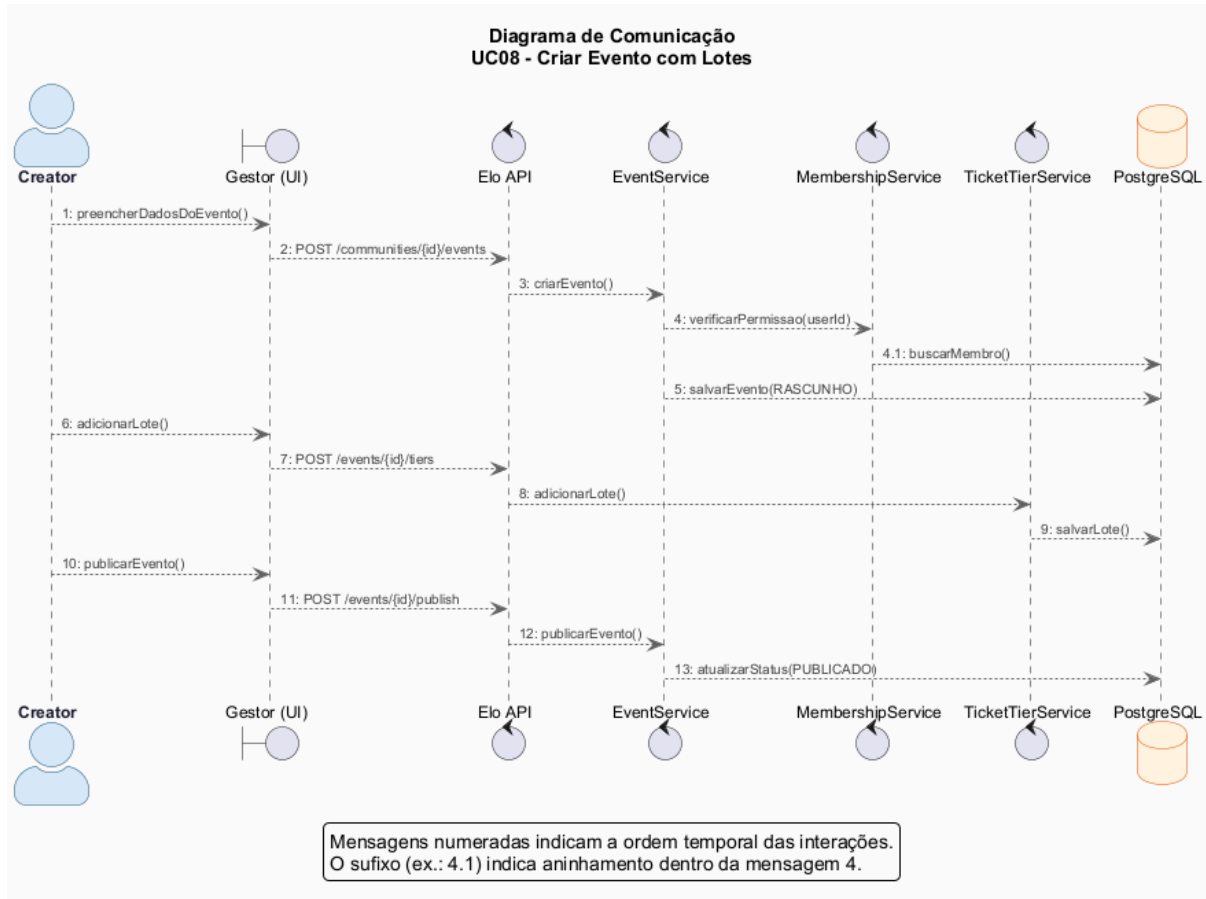


Figura 13: Diagrama de Comunicação: UC08 Criar Evento.

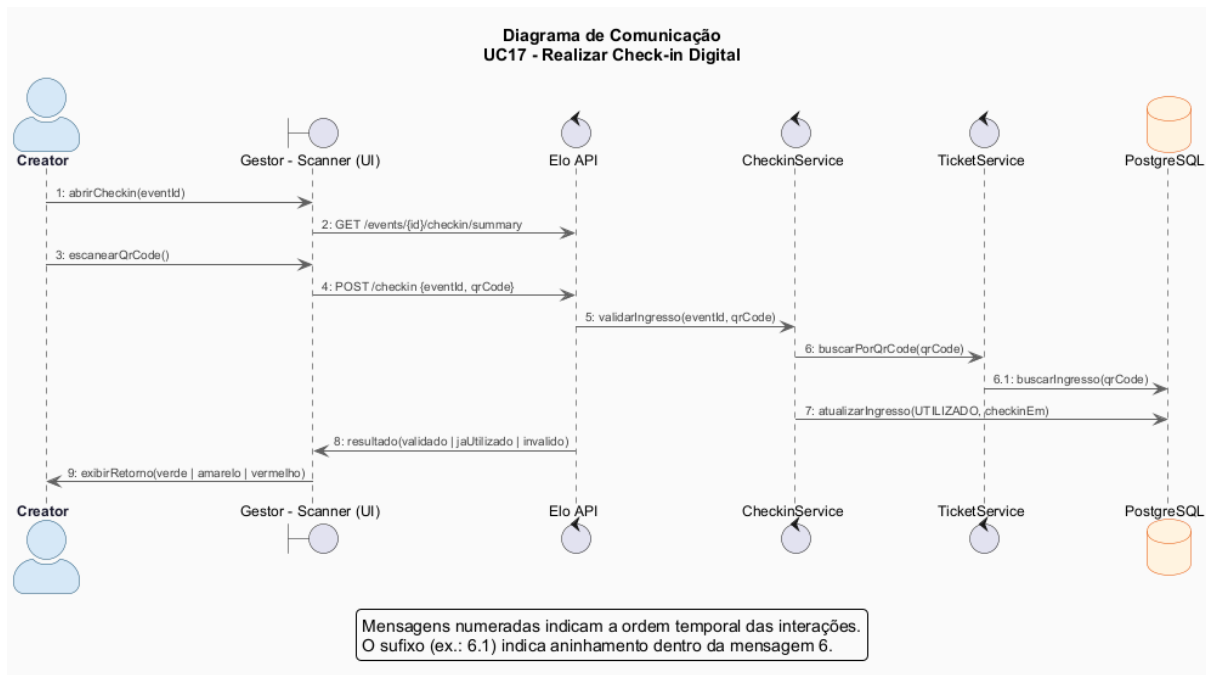


Figura 14: Diagrama de Comunicação: UC17 Check-in.

3.6 Diagramas de Estados

São modelados os ciclos de vida das entidades cujo comportamento é governado por estados: o Evento (Figura 15) e o Pedido em conjunto com o Ingresso (Figura 16).

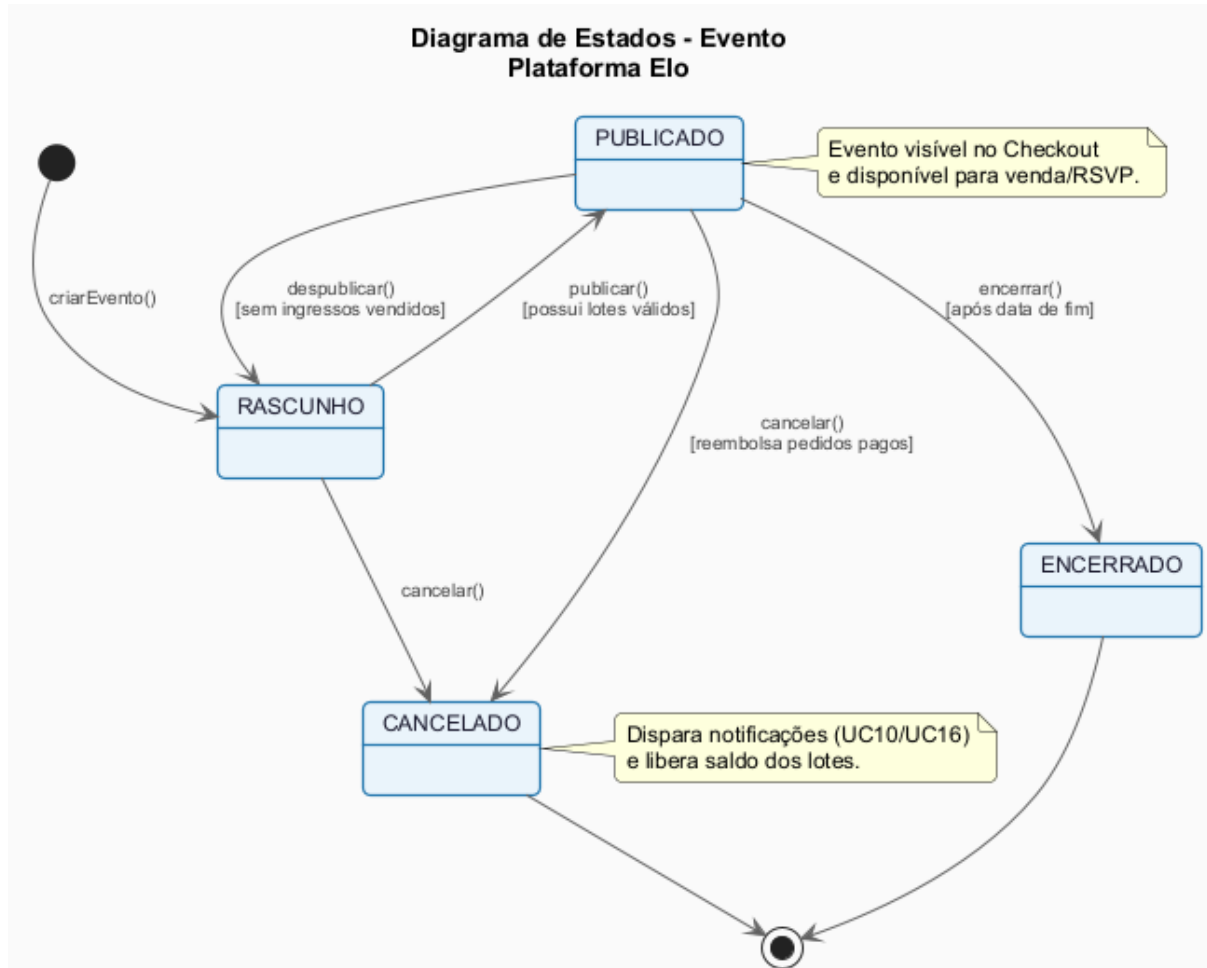


Figura 15: Diagrama de Estados: Evento.

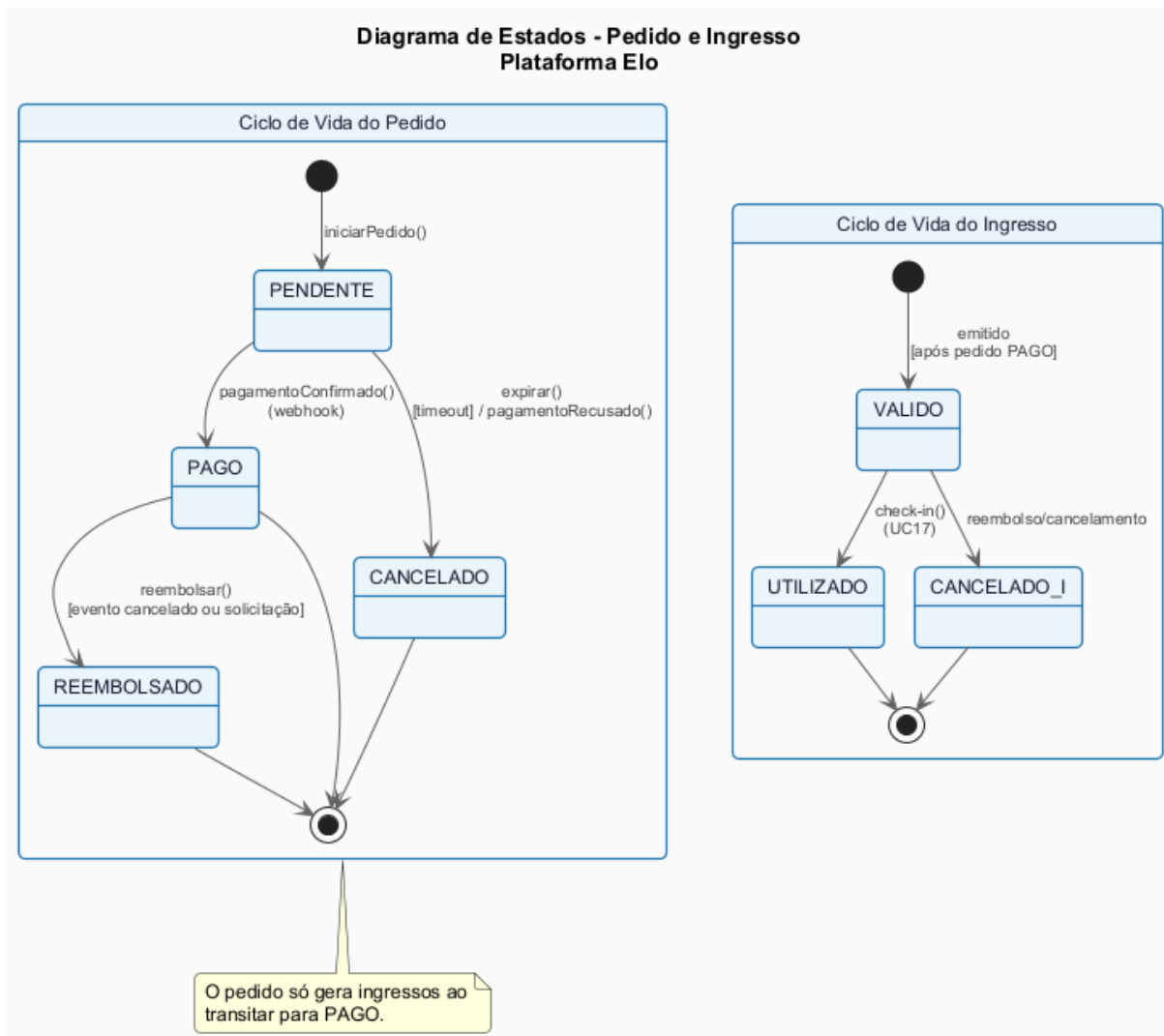


Figura 16: Diagrama de Estados: Pedido e Ingresso.

4. Modelos de Dados

O esquema de banco de dados do Elo é relacional, implementado em PostgreSQL 17 e gerenciado via Drizzle ORM. A Figura 17 apresenta o modelo entidade-relacionamento, com chaves primárias (PK), chaves estrangeiras (FK), restrições de unicidade e cardinalidades em notação *crow's foot*.

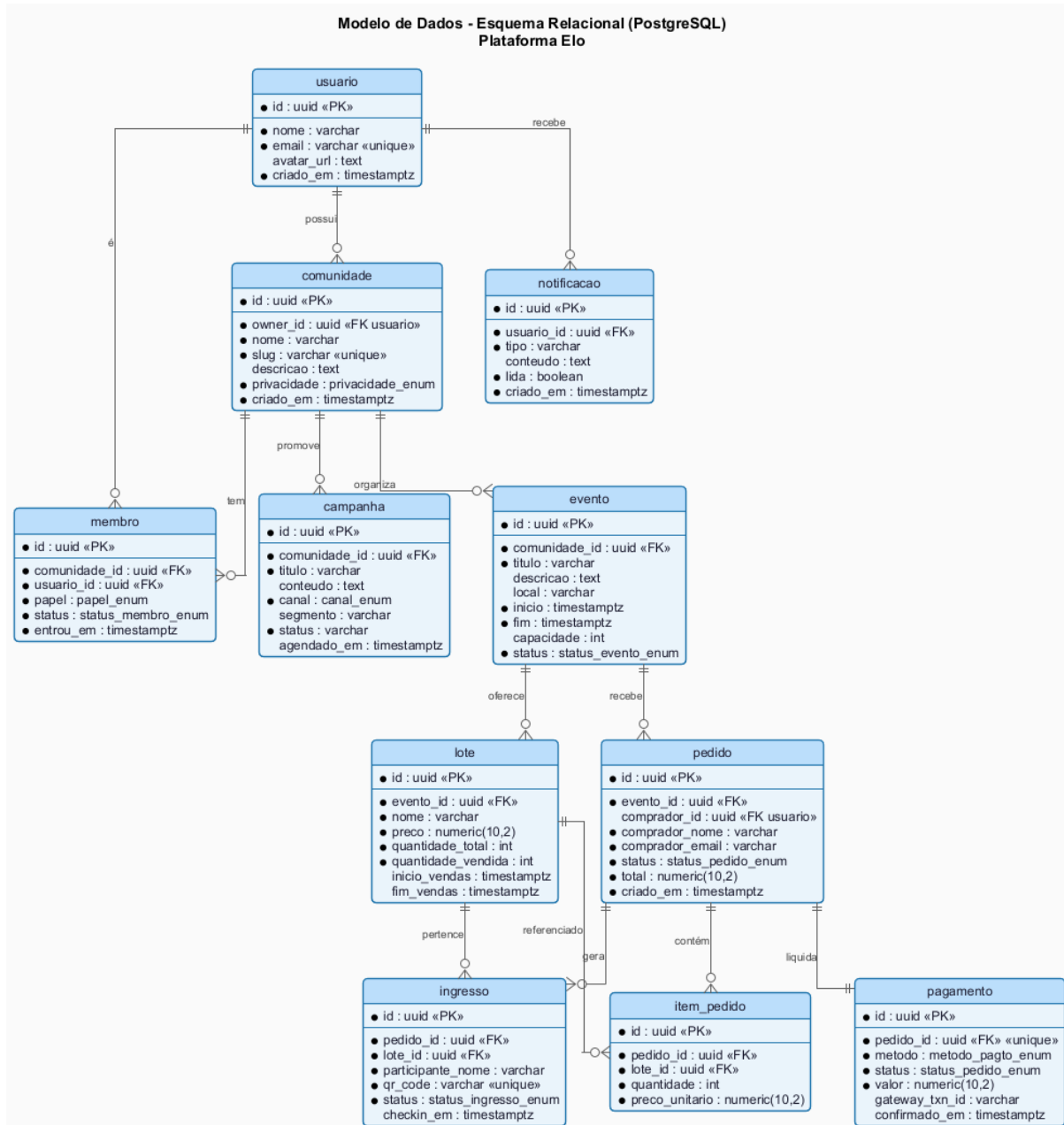


Figura 17: Modelo de Dados: esquema relacional (PostgreSQL).

Estratégia de mapeamento objeto-relacional. A correspondência entre o modelo de domínio (Seção 3.3) e o esquema relacional segue as convenções:

- cada *classe de entidade* é mapeada para uma **tabela**, e cada instância para uma **linha**, identificada por chave primária uuid;
- as *enumerações* do domínio são mapeadas para **tipos** enum nativos do PostgreSQL (ex.: `status_evento_enum`);
- as *associações um-para-muitos* (ex.: Comunidade–Evento) são representadas por uma **chave estrangeira** no lado “muitos”;
- as *composições* (ex.: Pedido–ItemPedido, Pedido–Ingresso) usam chave estrangeira com integridade referencial em cascata;

- a associação *um-para-um* Pedido–Pagamento é garantida por uma **restrição** `unique` sobre `pedido_id`;
- o controle de acesso multi-inquilino (*multi-tenant*) é reforçado por **Row-Level Security (RLS)** do Supabase, restringindo cada linha ao seu *tenant* (comunidade) de origem.